1. Bus Passenger Occupancy Simulation

Problem: Process boarding and alighting at each stop, calculate occupancy, and flag capacity violations.

Algorithm

1. Start and set the bus capacity and starting passenger count.
2. For each stop, read the number boarding and alighting.
3. Subtract alighting passengers, then add boarding passengers.
4. If occupancy is below 0 or above capacity, report a capacity violation.
5. Display occupancy after every stop and stop.

Flowchart (text form)

START → Read capacity and initial passengers → For each stop: read boarding/alighting → Update passengers = passengers – alighting + boarding → Check 0 ≤ passengers ≤ capacity → Display count/violation → More stops? (Yes: repeat; No: END)

Pseudocode

```
INPUT capacity, passengers, number_of_stops
FOR each stop
    INPUT boarding, alighting
    IF alighting > passengers
        PRINT "Invalid alighting count"
    ELSE
        passengers = passengers - alighting + boarding
        PRINT "Passengers:", passengers
        IF passengers > capacity OR passengers < 0
            PRINT "CAPACITY VIOLATION"
        ENDIF
    ENDIF
END FOR
```

Python code

```
capacity = int(input("Bus capacity: "))
passengers = int(input("Starting passengers: "))
stops = int(input("Number of stops: "))

for stop in range(1, stops + 1):
    boarding = int(input(f"Stop {stop} - boarding: "))
    alighting = int(input(f"Stop {stop} - alighting: "))

    if boarding < 0 or alighting < 0 or alighting > passengers:
        print("Invalid passenger count")
        continue

    passengers = passengers - alighting + boarding
    print("Passengers after stop:", passengers)

    if passengers > capacity:
        print("CAPACITY VIOLATION")
    else:
        print("Within capacity")
```

Sample output

```
Bus capacity: 40
Starting passengers: 20
Number of stops: 2
Stop 1 - boarding: 10
Stop 1 - alighting: 5
Passengers after stop: 25
Within capacity
Stop 2 - boarding: 20
Stop 2 - alighting: 3
Passengers after stop: 42
CAPACITY VIOLATION
```

2. Hackathon Team Formation and Ranking

Problem: Validate team sizes, total round scores, and rank teams. Ties are broken by higher score in the final round.

Algorithm

1. Read the minimum and maximum allowed team sizes.
2. For each team, read members and round-wise scores.
3. Validate the number of members.
4. Calculate the total score for each valid team.
5. Sort teams by total score descending, then final-round score descending.
6. Display totals and rankings.

Flowchart (text form)

START → Read team-size limits → Read each team's members and round scores → Validate size → Calculate total → Sort by total, then final-round score → Display ranking → END

Pseudocode

```
INPUT minimum_size, maximum_size
FOR each team
    INPUT team_name, members, round_scores
    IF minimum_size <= length(members) <= maximum_size
        total = SUM(round_scores)
        SAVE team_name, total, last(round_scores)
    ELSE
        PRINT "Invalid team size"
    ENDIF
END FOR
SORT valid teams by total DESCENDING, final-round score DESCENDING
DISPLAY ranked teams
```

Python code

```
min_size = 2
max_size = 4
teams = [
    ("Team A", ["Ana", "Ben", "Dev"], [15, 20, 18]),
    ("Team B", ["Ira", "Jay"], [20, 18, 15]),
    ("Team C", ["Kim", "Leo", "Mia", "Noa", "Omi"], [25, 20, 20])
]

valid = []
for name, members, scores in teams:
    if min_size <= len(members) <= max_size:
        valid.append((name, sum(scores), scores[-1]))
    else:
        print(name, "- invalid team size")

valid.sort(key=lambda item: (item[1], item[2]), reverse=True)
print("Team rankings:")
for rank, (name, total, final_score) in enumerate(valid, start=1):
    print(rank, name, "Total:", total, "Final round:", final_score)
```

Sample output

```
Team C - invalid team size
Team rankings:
1 Team A Total: 53 Final round: 18
2 Team B Total: 53 Final round: 15
```

3. Food Waste Monitoring System

Problem: Compare meals prepared, served, and left over each day; flag wastage above a set limit.

Algorithm

1. Set the acceptable wastage percentage.
2. For each day, input prepared and served meals.
3. Calculate leftovers = prepared – served.
4. If prepared is zero, avoid division by zero.
5. Calculate wastage percentage and flag days above the limit.
6. Display the daily summary.

Flowchart (text form)

START → Set wastage limit → Input prepared and served → Calculate leftovers → Calculate waste % (if prepared > 0) → Compare with limit → Display result → Next day? → END

Pseudocode

```
INPUT limit
FOR each day
    INPUT prepared, served
    IF prepared < 0 OR served < 0 OR served > prepared
        PRINT "Invalid data"
    ELSE IF prepared == 0
        waste_percent = 0
    ELSE
        leftovers = prepared - served
        waste_percent = leftovers / prepared * 100
    ENDIF
    IF waste_percent > limit
        PRINT "WASTE LIMIT EXCEEDED"
    ENDIF
    DISPLAY leftovers, waste_percent
END FOR
```

Python code

```
limit = 10
days = [
    ("Monday", 200, 180),
    ("Tuesday", 150, 145),
    ("Wednesday", 180, 150)
]

for day, prepared, served in days:
    if prepared < 0 or served < 0 or served > prepared:
        print(day, "- invalid data")
        continue
    leftovers = prepared - served
    waste_percent = (leftovers / prepared * 100) if prepared else 0
    print(f"{day}: leftovers={leftovers}, wastage={waste_percent:.1f}%")
    if waste_percent > limit:
        print("WASTE LIMIT EXCEEDED")
```

Sample output

```
Monday: leftovers=20, wastage=10.0%
Tuesday: leftovers=5, wastage=3.3%
Wednesday: leftovers=30, wastage=16.7%
WASTE LIMIT EXCEEDED
```

4. Multi-Branch Expense Matrix Analysis

Problem: Analyse monthly expenses stored as rows (branches) and columns (expense categories).

Algorithm

1. Store expenses in a nested list.
2. Calculate each branch total by adding its row.
3. Calculate each category total by adding values in the same column.
4. Calculate the overall total and average per cell.
5. Identify the branch with the highest total and category with the highest total.

Flowchart (text form)

START → Read expense matrix → Sum each row (branch totals) → Sum each column (category totals) → Calculate overall total and average → Find maximum branch/category → Display results → END

Pseudocode

```
INPUT expense_matrix
FOR each row
    branch_total = SUM(row)
END FOR
FOR each category column
    category_total = SUM(values in column)
END FOR
overall = SUM(all values)
average = overall / number_of_values
DISPLAY branch totals, category totals, overall, average
DISPLAY branch and category with maximum totals
```

Python code

```
branches = ["North", "South", "East"]
categories = ["Rent", "Stock", "Utilities"]
expenses = [
    [12000, 18000, 4000],
    [10000, 22000, 3500],
    [14000, 16000, 5000]
]

branch_totals = [sum(row) for row in expenses]
category_totals = [
    sum(expenses[r][c] for r in range(len(expenses)))
    for c in range(len(categories))
]
overall = sum(branch_totals)
average = overall / (len(expenses) * len(categories))

print("Branch totals:", dict(zip(branches, branch_totals)))
print("Category totals:", dict(zip(categories, category_totals)))
print("Overall expense:", overall)
print(f"Average per entry: {average:.2f}")
print("Largest branch:", branches[branch_totals.index(max(branch_totals))])
print("Largest category:", categories[category_totals.index(max(category_totals))])
```

Sample output

```
Branch totals: {'North': 34000, 'South': 35500, 'East': 35000}
Category totals: {'Rent': 36000, 'Stock': 56000, 'Utilities': 12500}
Overall expense: 104500
Average per entry: 11611.11
Largest branch: South
Largest category: Stock
```

5. Tuple-Based Transaction Audit

Problem: Check transaction tuples for duplicate IDs, invalid amounts, and suspicious sequences.

Algorithm

1. Read transaction tuples containing ID, amount, and type.
2. Track IDs already seen.
3. Flag repeated IDs.
4. Flag amounts that are zero or negative.
5. Flag consecutive repeated transaction types for the same ID when detected.
6. Print each issue and an audit summary.

Flowchart (text form)

START → Read transaction tuples → For each tuple: check duplicate ID → Check amount > 0 → Check transaction sequence → Record issues → Display audit report → END

Pseudocode

```
INPUT transactions
seen_ids = empty set
previous_type = empty value
FOR each (id, amount, type) in transactions
    IF id in seen_ids: FLAG duplicate
    ADD id to seen_ids
    IF amount <= 0: FLAG invalid amount
    IF type == previous_type: FLAG repeated type sequence
    previous_type = type
END FOR
DISPLAY all flags
```

Python code

```
transactions = [
    ("T01", 500, "PAYMENT"),
    ("T02", 250, "REFUND"),
    ("T01", 0, "PAYMENT"),
    ("T04", -50, "PAYMENT")
]

seen_ids = set()
previous_type = None
issues = []

for tx_id, amount, tx_type in transactions:
    if tx_id in seen_ids:
        issues.append(f"{tx_id}: duplicate transaction ID")
    seen_ids.add(tx_id)

    if amount <= 0:
        issues.append(f"{tx_id}: invalid amount {amount}")

    if tx_type == previous_type:
        issues.append(f"{tx_id}: suspicious consecutive type {tx_type}")
    previous_type = tx_type

print("Audit issues:")
if issues:
    for issue in issues:
        print("-", issue)
else:
    print("No issues found")
```

Sample output

```
Audit issues:
- T01: duplicate transaction ID
- T01: invalid amount 0
- T04: suspicious consecutive type PAYMENT
- T04: invalid amount -50
```

6. Student Group Formation with Score Balancing

Problem: Allocate students to groups while keeping group score totals as balanced as possible. This example uses a greedy rule.

Algorithm

1. Read student names and scores, and the required group count.
2. Sort students by score from highest to lowest.
3. Start with empty groups and totals of zero.
4. Assign each student to the group with the smallest current total.
5. Display each group, its members, and its total score.

Flowchart (text form)

START → Read students and group count → Sort scores descending → Choose group with smallest total → Assign next student → Update group total → More students? repeat → Display groups → END

Pseudocode

```
SORT students by score descending
CREATE required number of empty groups and zero totals
FOR each student
    target = group with smallest total
    ADD student to target
    UPDATE target total
END FOR
DISPLAY groups and totals
```

Python code

```
students = [
    ("Asha", 95), ("Bala", 88), ("Charan", 82),
    ("Divya", 78), ("Eshan", 72), ("Farah", 68)
]
group_count = 3
students.sort(key=lambda x: x[1], reverse=True)

groups = [[] for _ in range(group_count)]
totals = [0] * group_count

for student, score in students:
    target = totals.index(min(totals))
    groups[target].append((student, score))
    totals[target] += score

for i, group in enumerate(groups, start=1):
    print(f"Group {i}: {group}, Total score: {totals[i-1]}")
```

Sample output

```
Group 1: [('Asha', 95), ('Farah', 68)], Total score: 163
Group 2: [('Bala', 88), ('Eshan', 72)], Total score: 160
Group 3: [('Charan', 82), ('Divya', 78)], Total score: 160
```

7. Smart Canteen Order and Stock Management

Problem: Process customer orders, calculate bills, update stock, reject unavailable quantities, and report sales.

Algorithm

1. Set item prices and available stock.
2. For each order, check whether the item exists and quantity is positive.
3. If requested quantity exceeds stock, reject the order.
4. Otherwise calculate the bill, reduce stock, and add to total sales.
5. Display each order result, total sales, and remaining stock.

Flowchart (text form)

START → Set prices and stock → Read order → Validate item and quantity → Enough stock? → No: reject → Yes: calculate bill and update stock/sales → More orders? → Display summary → END

Pseudocode

```
SET price and stock dictionaries
sales = 0
FOR each order (item, quantity)
    IF item not found OR quantity <= 0
        REJECT order
    ELSE IF quantity > stock[item]
        PRINT "Insufficient stock"
    ELSE
        bill = price[item] * quantity
        stock[item] = stock[item] - quantity
        sales = sales + bill
        DISPLAY bill
    ENDIF
END FOR
DISPLAY sales and stock
```

Python code

```
price = {"Sandwich": 60, "Juice": 30, "Idli": 25}
stock = {"Sandwich": 10, "Juice": 8, "Idli": 20}
orders = [("Sandwich", 2), ("Juice", 3), ("Idli", 25), ("Idli", 4)]
total_sales = 0

for item, quantity in orders:
    if item not in stock or quantity <= 0:
        print("Rejected:", item, "- invalid order")
    elif quantity > stock[item]:
        print("Rejected:", item, "- insufficient stock")
    else:
        bill = price[item] * quantity
        stock[item] -= quantity
        total_sales += bill
        print(item, "Bill:", bill)

print("Total sales:", total_sales)
print("Remaining stock:", stock)
```

Sample output

```
Sandwich Bill: 120
Juice Bill: 90
Rejected: Idli - insufficient stock
Idli Bill: 100
Total sales: 310
Remaining stock: {'Sandwich': 8, 'Juice': 5, 'Idli': 16}
```

8. Library Borrowing and Fine Reconciliation

Problem: Calculate overdue fines, identify overdue records, and update book availability.

Algorithm

1. Set the allowed borrowing period and fine per late day.
2. For each borrowing record, read book ID, borrowed days, and return delay.
3. Calculate overdue days as the delay beyond the allowed period.
4. Calculate fine = overdue days × fine per day.
5. Mark books returned as available; otherwise mark them borrowed.
6. Display fines and availability.

Flowchart (text form)

START → Set allowed days and fine rate → Read each borrowing record → Calculate overdue days and fine → Check returned status → Update availability → Display report → END

Pseudocode

```
INPUT allowed_days, fine_per_day
FOR each record
    INPUT book_id, borrowed_days, returned
    overdue = MAX(0, borrowed_days - allowed_days)
    fine = overdue * fine_per_day
    IF overdue > 0: FLAG overdue
    IF returned: status = "Available"
    ELSE: status = "Borrowed"
    DISPLAY book_id, fine, status
END FOR
```

Python code

```
allowed_days = 14
fine_per_day = 5
records = [
    ("B101", 18, True),
    ("B102", 10, True),
    ("B103", 20, False)
]

for book_id, days, returned in records:
    overdue = max(0, days - allowed_days)
    fine = overdue * fine_per_day
    status = "Available" if returned else "Borrowed"
    print(f"{book_id}: overdue days={overdue}, fine=Rs.{fine}, status={status}")
    if overdue > 0:
        print("  OVERDUE RECORD")
```

Sample output

```
B101: overdue days=4, fine=Rs.20, status=Available
  OVERDUE RECORD
B102: overdue days=0, fine=Rs.0, status=Available
B103: overdue days=6, fine=Rs.30, status=Borrowed
  OVERDUE RECORD
```

9. Campus Transport Allocation

Problem: Calculate buses needed for each route based on passenger demand and bus capacity; identify unserved passengers.

Algorithm

1. Read bus capacity and passenger demand for each route.

2. Calculate buses required using ceiling(demand ÷ capacity).

3. Calculate seats provided and unserved passengers if the available bus count is limited.

4. Display buses required and any unserved demand.

Flowchart (text form)

START → Read capacity and route demand → Calculate buses needed = ceiling(demand/capacity) → Read buses available → Calculate served and unserved passengers → Display route summary → END

Pseudocode

```
INPUT capacity
FOR each route
    INPUT demand, buses_available
    required = CEILING(demand / capacity)
    seats = buses_available * capacity
    served = MIN(demand, seats)
    unserved = demand - served
    DISPLAY required, served, unserved
END FOR
```

Python code

```
import math

capacity = 40
routes = [
    ("Route A", 95, 3),
    ("Route B", 70, 1),
    ("Route C", 25, 1)
]

for route, demand, buses_available in routes:
    required = math.ceil(demand / capacity)
    served = min(demand, buses_available * capacity)
    unserved = demand - served
    print(f"{route}: buses required={required}, available={buses_available}, "
          f"served={served}, unserved={unserved}")
```

Sample output

```
Route A: buses required=3, available=3, served=95, unserved=0
Route B: buses required=2, available=1, served=40, unserved=30
Route C: buses required=1, available=1, served=25, unserved=0
```

10. Manufacturing Quality Control Matrix

Problem: Analyse production and defective units by product and batch; flag defect rates above a threshold.

Algorithm

1. Set the maximum acceptable defect percentage.
2. Store production and defective counts in a matrix.
3. For each batch, validate defective units do not exceed production.
4. Calculate defect percentage = defective ÷ production × 100.
5. Flag batches above the threshold and display the report.

Flowchart (text form)

START → Set defect threshold → Read production/defect matrix → For each batch validate counts → Calculate defect % → Compare with threshold → Flag if exceeded → Display report → END

Pseudocode

```
INPUT threshold
FOR each product and batch
    INPUT produced, defective
    IF produced <= 0 OR defective < 0 OR defective > produced
        PRINT "Invalid batch data"
    ELSE
        rate = defective / produced * 100
        DISPLAY rate
        IF rate > threshold
            PRINT "QUALITY THRESHOLD EXCEEDED"
        ENDIF
    ENDIF
END FOR
```

Python code

```
threshold = 5
batches = [
    ("Phone", "Batch 1", 1000, 30),
    ("Phone", "Batch 2", 800, 56),
    ("Tablet", "Batch 1", 600, 18),
    ("Tablet", "Batch 2", 500, 10)
]

for product, batch, produced, defective in batches:
    if produced <= 0 or defective < 0 or defective > produced:
        print(product, batch, "- invalid data")
        continue
    rate = defective / produced * 100
    print(f"{product} {batch}: defect rate={rate:.2f}%")
    if rate > threshold:
        print("  QUALITY THRESHOLD EXCEEDED")
```

Sample output

```
Phone Batch 1: defect rate=3.00%
Phone Batch 2: defect rate=7.00%
  QUALITY THRESHOLD EXCEEDED
Tablet Batch 1: defect rate=3.00%
Tablet Batch 2: defect rate=2.00%
```